

Lec22

Problems

Earlier websites worked like this:

1. User clicks button
2. Browser sends request to server
3. Entire page reloads
4. New data appears

Example:

- Fill form
- Submit
- Whole page refreshes

Bad because:

- Slow
- Poor user experience

What is AJAX?

AJAX stands for:

Asynchronous

JavaScript

And

XML

Even though AJAX name contains XML, today mostly JSON is used.

AJAX allows:

Sending and receiving data from server without reloading webpage.

Example

- Instagram likes
- Gmail inbox refresh
- Live search
- Weather apps
- Chat apps

Example:

You click “Load Users”

Data comes from server

Page updates instantly

No page refresh

That is AJAX.

Synchronous vs Asynchronous

Synchronous

Tasks execute one by one.

Example:

```
console.log("Task 1");  
console.log("Task 2");  
console.log("Task 3");
```

Output:

Task 1

Task 2

Task 3

Asynchronous

Some tasks take time:

- API request
- Timer
- Database query

Example:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Data Loaded");  
}, 2000);
```

```
console.log("End");
```

Output:

Start

End

Data Loaded

Because request takes time.

Why AJAX?

Benefits:

- No page reload
- Faster UI
- Better user experience
- Real-time updates

Used in:

- Social media
- E-commerce
- Dashboards
- Banking apps

XMLHttpRequest (XHR)

Before Fetch API, AJAX used **XMLHttpRequest**.

It is a JavaScript object used to communicate with servers.

Create object:

```
let xhr = new XMLHttpRequest();
```


Basic Steps of XHR

1. Create object
2. Open request
3. Send request
4. Receive response

Example

```
let xhr = new XMLHttpRequest();

xhr.open("GET", "https://jsonplaceholder.typicode.com/users");

xhr.onload = function() {
  console.log(xhr.responseText);
};

xhr.send();
```

Explanation:

Create Object

```
let xhr = new XMLHttpRequest();
```

Open Request

```
xhr.open("GET", url);
```

Parameters:

- Method (GET/POST)
- URL

Send Request

```
xhr.send();
```

Request goes to server.

Receive Response

```
xhr.onload = function() {  
  console.log(xhr.responseText);  
};
```

Server response received.

HTTP Methods

When talking to server, methods matter.

GET

Retrieve data.

Example:

- Get users
- Get products

GET /users

POST

Send new data.

Example:

- Signup
- Add product

POST /users

PUT

Update data.

DELETE

Delete data.

Response Data

Server sends response.

Usually in JSON.

Example:

```
[  
  {  
    "id": 1,  
    "name": "John"  
  }  
]
```

What is API?

API stands for:

Application Programming Interface

Simple meaning:

A bridge that allows two systems to communicate.

Example:

Your frontend wants weather data.

Instead of collecting weather manually,
you ask Weather API.

Flow:

Frontend → API → Server/Database

Real Example

Food delivery app:

Frontend asks:

GET /restaurants

API returns:

```
[  
  {  
    "name": "Pizza Hut"  
  }  
]
```

Frontend displays restaurants.

JSON

Most APIs return JSON.

JSON = JavaScript Object Notation

Example:

```
{  
  "name": "Naman",  
  "age": 22  
}
```

Looks like JS object but keys are strings.

Convert JSON ↔ JavaScript Object

JSON → Object

JSON.parse()

Object → JSON

JSON.stringify()

Example:

```
let user = '{"name":"Naman"}';
```

```
let obj = JSON.parse(user);
```

```
console.log(obj.name);
```

Fetch API (Modern Way)

Today we mostly use Fetch API instead of XHR.

Why?

- Cleaner
- Easier
- Promise-based

Basic syntax:

```
fetch(url)
```

Fetch Example

```
fetch("https://jsonplaceholder.typicode.com/users")  
  .then(response => response.json())  
  .then(data => console.log(data));
```

Explanation:

Step 1

Send request:

```
fetch(url)
```

Step 2

Convert response to JSON:

```
response.json()
```

Step 3

Use data:

```
.then(data => console.log(data))
```

Fetching External Data and Showing in UI

```
<ul id="users"></ul>
```

```
async function getUsers() {  
  let response = await fetch("https://jsonplaceholder.typicode.com/users");  
  let users = await response.json();
```

```
  let list = document.getElementById("users");
```

```
  users.forEach(user => {  
    let li = document.createElement("li");  
    li.innerText = user.name;  
    list.appendChild(li);  
  });  
}
```

```
getUsers();
```

Common API Status Code

Code	Meaning
200	Success
201	Created
400	Bad Request
404	Not Found
500	Server Error

XHR vs Fetch

XMLHttpRequest	Fetch API
Old	Modern
Complex	Easy
Callback-based	Promise-based

Task

Fetch API data and display it on the webpage